```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# =========================
# 1. Load Dataset
# =========================
df = pd.read_csv("/content/spam_or_not_spam.csv")

print("Dataset Preview:")
print(df.head())
print(df.tail())

# Handle NaN values in the 'email' and 'label' columns
df.dropna(subset=['email', 'label'], inplace=True)

# Assuming dataset has columns: 'email' and 'label'
# (Change column names if needed)

X = df['email']
y = df['label']

# =========================
# 2. Train-Test Split
# =========================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =========================
# 3. TF-IDF Feature Extraction
# =========================
vectorizer = TfidfVectorizer(stop_words='english')
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

X_train_tfidf = X_train_tfidf.toarray()
X_test_tfidf = X_test_tfidf.toarray()

# =========================
# 4. Single Neuron Model
# =========================
epochs = 40
learning_rate = 0.05

w = np.zeros(X_train_tfidf.shape[1])
b = 0

loss_list = []
accuracy_list = []

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# =========================
# 5. Training
# =========================
for epoch in range(epochs):

    total_loss = 0

    for x, y_true in zip(X_train_tfidf, y_train):

        # Forward Pass
        z = np.dot(x, w) + b
        y_pred = sigmoid(z)

        # Loss (Binary Cross Entropy)
        loss = -(y_true*np.log(y_pred+1e-9) +
                (1-y_true)*np.log(1-y_pred+1e-9))
        total_loss += loss
```

```
            # Backward Pass
            error = y_pred - y_true
            dw = error * x
            db = error

            # Update
            w -= learning_rate * dw
            b -= learning_rate * db

        # Store loss
        loss_list.append(total_loss / len(X_train_tfidf))

        # Calculate training accuracy
        preds = sigmoid(np.dot(X_train_tfidf, w) + b)
        preds = [1 if i >= 0.5 else 0 for i in preds]
        acc = accuracy_score(y_train, preds)
        accuracy_list.append(acc)

        print(f"Epoch {epoch+1}: Loss={loss_list[-1]:.4f}, Accuracy={acc:.4f}")

    # ========================
    # 6. Testing
    # ========================
    test_preds = sigmoid(np.dot(X_test_tfidf, w) + b)
    test_preds = [1 if i >= 0.5 else 0 for i in test_preds]

    # ========================
    # 7. Evaluation Metrics
    # ========================
    print("\nEvaluation on Test Data:")
    print("Accuracy:", accuracy_score(y_test, test_preds))
    print("Precision:", precision_score(y_test, test_preds))
    print("Recall:", recall_score(y_test, test_preds))
    print("F1 Score:", f1_score(y_test, test_preds))

    # ========================
    # 8. Visualization
    # ========================
    plt.figure()
    plt.plot(range(1, epochs+1), loss_list)
    plt.title("Loss vs Epochs")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.show()

    plt.figure()
    plt.plot(range(1, epochs+1), accuracy_list)
    plt.title("Accuracy vs Epochs")
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy")
    plt.show()
```